SIMATS ENGINEERING

CO2_AT2 Reverse Engineering Task

COURSE NAME: Software Testing **COURSE CODE: CSA37**

COURSE OUTCOME COVERED

CO2: *Design effective test cases using black-box, white-box testing techniques and to control methods flow.*

Assessment Tool Weightage: 30%

Sl. No	Questions	Bloom's Level
1	A partially tested ATM withdrawal module is provided where only input-output behavior is known (e.g., withdrawal limits, balance validation, error messages). Reverse engineer the system by analyzing the observed behavior and reconstructing the underlying test case design strategy. Decompose the system into functional components and identify possible equivalence classes and boundary conditions. Infer the black box testing techniques used and evaluate their adequacy. Reconstruct an improved set of test cases covering all critical scenarios. Document your analysis with structured explanation.	5
2	A software module's execution traces and code snippets are given, but no test documentation is available. Reverse engineer the white box testing approach by constructing the control flow graph from the given logic. Identify all decision points, loops, and independent execution paths. Evaluate the existing test adequacy in terms of coverage criteria. Reconstruct a complete path testing strategy ensuring full coverage. Present your findings with clear reasoning and structured representation.	5
3	A web application login and transaction system has been tested using unknown techniques, but only defect logs and test results are available. Reverse engineer the testing approach by analyzing defect patterns and identifying which black box and white box techniques were likely used. Decompose the system into testable components and evaluate gaps in testing coverage. Reconstruct a comprehensive testing strategy integrating equivalence partitioning, boundary value analysis, and path testing. Justify your reconstructed approach with logical reasoning and structured documentation.	5

4	A legacy inventory management system provides only a set of successful and failed transaction logs regarding stock updates and low-stock alerts. Reverse engineer the business logic by analyzing these logs to identify the hidden boundary conditions and equivalence partitions. Decompose the update process into functional units and reconstruct a black box test design that addresses the gaps found in the logs. Justify your selection of test cases based on the inferred logic.	5
5	You are provided with a compiled .exe file of a library management module and its corresponding CPU/Memory execution profile during a stress test. Reverse engineer the white box testing requirements by mapping execution spikes to potential loops and decision clusters in the logic. Construct a conceptual control flow graph (CFG) that represents this behavior. Reconstruct a path testing strategy that targets these resource-intensive paths and evaluate its impact on system stability.	6
6	A mobile payment gateway's API responses (JSON) for various transaction states—Pending, Success, Reversal, and Timeout—are given. Reverse engineer the underlying state machine by analyzing the transition triggers and data constraints. Identify the implicit state transition testing technique used. Reconstruct a comprehensive state-based test suite that ensures all legal and illegal transitions are covered. Document your analysis with a structured state-transition diagram.	5
7	An encrypted communication module provides only "black-box" behavior through its input parameters and encrypted outputs. Reverse engineer the validation logic by performing "fuzzing" analysis on the input fields to identify error-handling boundaries. Decompose the module into data validation and encryption components. Reconstruct an improved set of test cases using Boundary Value Analysis (BVA) and justify how your design prevents buffer overflow or injection vulnerabilities.	5
8	A set of legacy unit test results for an insurance premium calculator shows 100% statement coverage but fails to catch defects in multi-condition "if" statements. Reverse engineer the original white box approach and identify the inadequacy in the coverage criteria used. Construct a new CFG and identify all independent execution paths. Reconstruct a path testing strategy that achieves 100% Branch and Path coverage. Explain how this reconstruction reduces the risk of calculation errors.	5

9	A cloud-based file synchronization tool provides user activity logs and a list of "lost file" defect reports. Reverse engineer the synchronization algorithm's logic by analyzing the defect patterns related to file size, type, and network speed. Infer the likely Equivalence Class Partitioning (ECP) used by the original developers. Reconstruct a revised testing strategy that integrates BVA with stress testing. Justify how your reconstructed approach prevents future data loss.	5
10	You are given a sequence of database queries (SQL) generated by an e-commerce "Search and Filter" module, but no source code. Reverse engineer the search logic by analyzing the query parameters and the resulting dataset. Identify the decision-making logic used for complex filtering (e.g., AND/OR/NOT). Reconstruct a Decision Table based testing strategy that covers all possible search combinations. Evaluate the completeness of your reconstructed model.	5

Code of Conduct:

I AATHISRI P(192521252) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:
AATHISRI P(192521252)

1. ATM Withdrawal Module – Reverse Engineering Black-Box Testing

Introduction: Reverse engineering identifies the testing strategy of an existing system by observing its inputs, outputs and error messages without examining its source code.

1. Functional Decomposition

The ATM withdrawal module can be divided into:

- Input validation: checks the withdrawal amount.
- Balance validation: verifies sufficient funds.
- Limit validation: checks daily and per-transaction limits.
- Transaction processing: deducts the amount.
- Error handling: displays appropriate error messages.

2. Equivalence Class Partitioning (ECP)

Divide inputs into valid and invalid classes. Assume the minimum withdrawal is ₹100 and the maximum is ₹25,000.

Input	Expected result
Negative amount	Reject
Zero amount	Reject
Valid amount	Accept if balance is sufficient
Amount above limit	Reject
Amount exceeding balance	Reject

3. Boundary Value Analysis (BVA)

Test ₹99, ₹100, ₹101 and ₹24,999, ₹25,000, ₹25,001 to detect boundary errors.

4. Improved Test Cases

- Valid withdrawal with sufficient balance.
- Withdrawal with insufficient balance.
- Invalid PIN and blocked card.

- Daily withdrawal limit exceeded.
- Network failure during withdrawal.
- Duplicate transaction and cash-dispensing failure.

5. Evaluation

Observed behavior helps infer whether ECP, BVA or other black-box techniques were used. However, the exact original technique cannot be confirmed from outputs alone.

Conclusion: Combining ECP, BVA and transaction-failure testing improves withdrawal accuracy, prevents invalid transactions and strengthens ATM reliability.

2. Software Module – CFG and Path Testing

Introduction: Reverse engineering white-box testing involves reconstructing program control flow from code snippets and execution traces to identify missing test paths.

1. Construct a Control Flow Graph (CFG)

A CFG contains nodes representing statements or blocks and edges representing possible execution flows.

Consider a module that validates input, checks a condition and processes the result.

Start
Validate input
Input valid?

Yes

Process data

No

Display error
End

2. Identify Decision Points and Paths

The main paths are:

- Path 1: Start → Validate → Valid → Process → End.
- Path 2: Start → Validate → Invalid → Error → End.

If loops are present, also test loop entry, termination and repeated execution.

3. Cyclomatic Complexity

The number of independent paths can be estimated using:

\[V(G)=E-N+2P \]

Where \(E\) is the number of edges, \(N\) is the number of nodes and \(P\) is the number of connected components. For one connected graph, \(V(G)=E-N+2\).

4. Improved Testing Strategy

- Identify all decisions and loops.
- Design tests for each independent path.
- Measure statement and branch coverage.
- Test loop boundaries and exceptional conditions.
- Add test cases for uncovered paths.

Conclusion: CFG reconstruction and basis path testing help reveal missing execution paths .

3. Web Login and Transaction System – Hybrid Testing

Introduction: Defect logs and test results can be used to infer the testing methods previously applied to a web login and transaction system.

1. Functional Decomposition

Divide the system into:

- Login validation.
- Authentication and session management.
- Transaction input validation.
- Balance and authorization checks.
- Transaction processing.
- Error handling and logging.

2. Reverse Engineer Existing Testing

- Defects near minimum or maximum values may indicate insufficient BVA.
- Errors involving similar input categories may indicate incomplete ECP.
- Defects in conditional logic may indicate missing branch or path testing.
- Authentication failures may indicate inadequate negative testing.

These patterns suggest possible gaps but do not prove which techniques were originally used.

3. ECP and BVA

Test valid and invalid usernames, passwords, transaction amounts and account details. Test values immediately below, at and above transaction limits.

4. Path Testing

Construct a CFG for login and transaction processing. Test paths for successful login, invalid credentials, insufficient funds, unauthorized transactions and transaction failure.

5. Improved Hybrid Strategy

- Apply ECP and BVA to all important inputs.
- Test both true and false decision branches.
- Verify session expiration and authorization.
- Test rollback when a transaction fails.
- Automate regression testing and monitor coverage.

Conclusion: Combining black-box and white-box techniques detects both input-related and internal logic

defects, improving security and transaction reliability.

4. Inventory Management System – Reverse Engineering Business Logic

Introduction: Stock update logs and low-stock alerts can reveal hidden business rules, validation boundaries and defects in an inventory management system.

1. Functional Decomposition

The system consists of:

- Stock input validation.
- Current stock retrieval.
- Quantity update.
- Minimum-stock comparison.
- Low-stock alert generation.
- Transaction logging.

2. Identify Equivalence Classes

For stock quantity, classify inputs as:

- Positive quantity.
- Zero quantity.
- Negative quantity.
- Quantity exceeding permitted limits.
- Invalid or missing quantity.

3. Boundary Value Analysis

Assume the low-stock threshold is 10 units.

Stock quantity	Expected result
-1	Reject invalid quantity
0	Apply zero-stock policy
9	Generate low-stock alert
10	Apply threshold rule

11	No low-stock alert

The exact behavior at the threshold must follow the recorded business rules.

4. Reconstructed Test Cases

- Add stock to existing inventory.
- Reduce stock without exceeding available quantity.
- Attempt a negative stock update.
- Test quantities around the alert threshold.
- Verify alerts after stock changes.
- Test simultaneous updates and database failures.

5. Improved Strategy

Compare successful and failed logs with expected results. Identify untested boundaries, add ECP and BVA cases, and verify stock consistency after every transaction.

Conclusion: Reverse engineering transaction logs helps reconstruct inventory rules and develop systematic tests for stock accuracy and alert reliability.

5. Library Management Module – Resource-Based Path Testing

Introduction: CPU and memory execution profiles from a compiled executable can help identify resource-intensive operations, although they do not directly reveal the source code or exact control flow.

1. Analyze Execution Profiles

Identify operations associated with:

- High CPU usage.
- Increasing memory consumption.
- Repeated processing.
- Slow responses under heavy workloads.

These may indicate loops, repeated searches, nested processing or inefficient resource management.

2. Construct a Conceptual CFG

Start
Read library request
Records available?

Yes

Search and process records

No

Return empty result
End

3. Path Testing Strategy

- Test empty, small and large library datasets.
- Test successful searches and missing records.
- Test repeated searches and nested record-processing paths.
- Test maximum supported data size.
- Measure execution time and memory consumption.
- Check for memory leaks and unbounded loops.

4. Cyclomatic Complexity

Use \(V(G)=E-N+2P\) when the CFG can be reconstructed. Select independent paths based on the graph, prioritizing resource-intensive operations.

5. Expected Impact

Targeted path testing helps expose slow loops, excessive memory usage and termination problems. Stress and load testing further evaluate stability under heavy workloads.

Conclusion: Combining profile analysis, conceptual CFG reconstruction, path testing and stress testing improves the stability and performance of the library module.

6. Mobile Payment Gateway – State Transition Testing

Introduction: A mobile payment gateway processes transactions through Pending, Success, Reversal and Timeout states. Reverse engineering API responses helps reconstruct the underlying state machine.

1. Identify the States

- Pending: Transaction is being processed.
- Success: Payment is completed.
- Reversal: A previous payment is cancelled or its effect is reversed.
- Timeout: No response is received within the permitted time.

2. State Transition Diagram

Pending
Success
Timeout
Reversal

Possible transitions depend on payment rules and gateway responses.

3. State Transition Test Cases

Current state	Event	Expected result
Pending	Payment confirmed	Success
Pending	Response timeout	Timeout or pending reconciliation
Success	Valid reversal request	Reversal
Success	Duplicate payment request	No duplicate charge
Timeout	Late payment response	Reconcile actual payment status
Reversal	Duplicate reversal request	No duplicate refund

4. Improved Test Strategy

- Test every legal transition.
- Test illegal transitions and duplicate events.
- Verify transaction IDs, amounts and status fields in JSON responses.

- Test delayed responses and network interruptions.
- Check that retries do not create duplicate charges.
- Verify consistency between the gateway and merchant records.

Conclusion: State Transition Testing validates transaction sequences, prevents duplicate payments and improves the reliability of payment processing.

7. Encrypted Communication Module – Fuzzing and BVA

Introduction: An encrypted communication module exposes input parameters and encrypted outputs without revealing its internal logic. Fuzzing and BVA can help discover input-validation defects and security weaknesses.

1. Functional Decomposition

Divide the module into:

- Input validation.
- Data-length and format checking.
- Encryption processing.
- Error handling.
- Output generation.

2. Fuzzing Analysis

Provide unexpected or malformed inputs to identify crashes, abnormal responses and validation failures. Examples include:

- Empty inputs.
- Very long strings.
- Invalid encodings.
- Unexpected special characters.
- Missing parameters.
- Malformed binary data.

3. Boundary Value Analysis

If the maximum permitted input length is \(L\), test:

Input length	Expected result
0	Accept or reject according to requirements
1	Validate correctly
\(L-1\)	Accept
\(L\)	Accept if inclusive

\(L+1\) Reject safely

4. Improved Security Testing

- Validate input lengths before processing.
- Test malformed and oversized inputs.
- Check for buffer overflow, crashes and memory errors.
- Use parameterized handling and safe memory operations.
- Test against injection attacks where inputs reach interpreters or commands.
- Verify that error messages do not expose sensitive information.
- Confirm that valid inputs produce correct encryption outputs.

Conclusion: Fuzzing explores unexpected inputs, while BVA targets length limits. Together, they improve input validation and help detect vulnerabilities, though security cannot be guaranteed by these techniques alone.

8. Insurance Premium Calculator – Branch and Path Testing

Introduction: The insurance premium calculator achieves 100% statement coverage but still contains defects in multi-condition if statements. This indicates that statement coverage alone is insufficient.

1. Limitation of Existing Testing

Statement coverage confirms that each statement executes at least once. It does not ensure that each decision outcome or each individual Boolean condition has been tested.

2. Construct a CFG

Consider an illustrative premium calculation:

if (AgeEligible AND NoClaims)

- True branch: Apply the eligible premium discount.
- False branch: Calculate the standard premium.

Start
Age eligible AND no claims?

True

Apply discount

False

Standard premium
Return premium

3. Independent Test Cases

Age eligible	No claims	Expected path
True	True	Discount
True	False	Standard premium
False	True	Standard premium
False	False	Standard premium

4. Improved Testing Strategy

- Achieve branch coverage by testing true and false decision outcomes.
- Exercise each independent path in the CFG.
- Test premium boundaries for age, coverage and claim history.
- Verify calculated premiums against expected values.
- Measure branch coverage and perform regression testing.

5. Path Coverage Consideration

For simple finite control flow, all feasible paths can be enumerated. In larger systems with loops, exhaustive path coverage may be impractical; independent paths and critical paths should be prioritized.

Conclusion: Branch and path testing reveal logical defects that statement coverage can miss, reducing the risk of incorrect insurance premium calculations.

9. Cloud File Synchronization – ECP, BVA and Stress Testing

Introduction: Lost-file defects may be associated with file size, file type and network speed. Reverse engineering user activity logs can reveal gaps in the original input partitions and synchronization tests.

1. Functional Decomposition

The system includes:

- File selection and validation.
- File-size and type checking.
- Upload and download processing.
- Network transfer and retry handling.
- Conflict resolution.
- File integrity verification.

2. Equivalence Class Partitioning

Input	Equivalence classes
File size	Empty, small, valid large, oversized
File type	Supported, unsupported, corrupted
Network speed	Stable, slow, intermittent, disconnected
File status	New, modified, deleted, conflicting

3. Boundary Value Analysis

If the maximum supported file size is \(S\), test:

- \(S-1\): just below the limit.
- \(S\): exactly at the limit.
- \(S+1\): just above the limit.

Also test empty files, large numbers of files and transfer interruptions.

4. Stress Testing

- Synchronize many files simultaneously.
- Test large files over slow networks.
- Interrupt and restore connectivity.
- Test repeated retries and concurrent file modifications.
- Verify checksums and file counts after synchronization.

5. Improved Testing Strategy

Combine ECP, BVA and stress testing. Maintain logs of synchronization states, detect incomplete transfers, and ensure failed operations can resume without data loss.

Conclusion: ECP and BVA improve input coverage, while stress testing exposes failures under heavy workloads and poor networks. File-integrity checks further reduce the risk of data loss.

10. E-Commerce Search and Filter – Decision Table Testing

Introduction: SQL queries and resulting datasets can be analyzed to reconstruct the decision-making logic of an e-commerce search and filter module.

1. Functional Decomposition

Divide the module into:

- Keyword search.
- Category filtering.
- Price-range filtering.
- Availability filtering.
- Sorting.
- Result generation.

2. Identify Logical Operators

- AND: All selected conditions must be satisfied.
- OR: At least one selected condition must be satisfied.
- NOT: Excludes results matching a specified condition.

For example, a query may return products in a selected category AND within a price range.

3. Decision Table

Assume the search filters include category, price and availability.

Rule	Category matches	Price matches	Available	Expected result
R1	Yes	Yes	Yes	Include product
R2	Yes	No	Yes	Exclude product

R3	No	Yes	Yes	Exclude product
R4	Yes	Yes	No	Exclude product
R5	Yes	Yes	Yes, with OR filter	Apply defined OR logic
R6	Any	Any	Any, with NOT condition	Exclude matching products

4. Reconstructed Testing Strategy

- Test each filter independently.
- Test combinations of two and three filters.
- Verify AND, OR and NOT behavior.
- Test empty searches and no-result searches.
- Test minimum and maximum price boundaries.
- Compare actual results with expected datasets.
- Check sorting, pagination and invalid parameters.
- Use parameterized queries to prevent SQL injection.

5. Evaluate Completeness

Decision table testing covers identified combinations, but the table must reflect the actual business rules. For numerous filters, pairwise testing or risk-based combination testing can reduce test count, while critical combinations receive explicit tests.

Conclusion: Reverse engineering SQL queries and using decision tables reconstructs search logic systematically. The resulting tests improve filtering accuracy, result completeness and application security.

RUBRICS FOR CALCULATION:3

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Understanding of System	20%	Demonstrates clear and in-depth understanding of the system/product and its functionality	Shows good understanding with minor gaps	Partial understanding; misses key aspects	Lacks understanding of the system/product
Decomposition	25%	Effectively breaks down the system into components with detailed analysis	Decomposes system with minor gaps in analysis	Limited or incomplete decomposition	Unable to analyze or break down system
Identification of Components	20%	Accurately identifies all components and explains their functions clearly	Identifies most components with minor errors	Identifies few components; explanations are weak	Unable to identify components or functions
Reconstruction	20%	Successfully reconstructs or models the system with accuracy and logic	Reconstruction is mostly correct with minor issues	Partial reconstruction; lacks clarity	Unable to reconstruct or model system
Documentation	15%	Well-organized, clear documentation with diagrams and structured explanation	Documentation is adequate with minor issues	Poorly organized or incomplete documentation	No proper documentation or presentation